

boucle

title: “Case 8 — Boucler” subtitle: “Passer de $Y \sim X$ à $Y \sim$ tous les X ” author: “SeRiouS”
format: beamer: aspectratio: 169 theme: Madrid colortheme: seahorse fontsize: 10pt execute:
echo: true eval: false

Objectif de la case

Jusqu’ici, nous avons surtout travaillé sur des analyses uniques.

Par exemple :

```
intention_achat ~ prix_percu
```

Dans cette case, l’objectif est de comprendre comment passer d’une analyse isolée à une série d’analyses répétées.

On veut donc passer de :

```
Y ~ X
```

à :

```
Y ~ X1  
Y ~ X2  
Y ~ X3  
...
```

Idée centrale

L'automatisation consiste à identifier ce qui change et ce qui reste identique.

Ici :

- la variable à expliquer reste la même ;
- la méthode reste la même ;
- seule la variable explicative change.

On peut donc écrire une fonction, puis l'appliquer à plusieurs variables.

La variable à expliquer

La variable à expliquer est définie dans un objet :

```
y <- "intention_achat"
```

Cette écriture est importante.

On ne code plus directement la variable dans la formule.

On la stocke dans un objet que l'on pourra réutiliser.

Les variables explicatives

Les variables explicatives sont stockées dans un vecteur :

```
variables_x <- c(  
  "prix_percu",  
  "plaisir",  
  "naturalite",  
  "confiance",  
  "ancrage_local",  
  "usage_numerique",  
  "sensibilite_env"  
)
```

Ce vecteur contient les différentes variables X que l'on veut tester.

Ce que permet cette organisation

On sépare clairement :

Élément	Objet R
Variable à expliquer	<code>y</code>
Variables explicatives	<code>variables_x</code>
Méthode d'analyse	<code>FactoMineR::LinearModel()</code>

Cette séparation rend le code plus facile à automatiser.

Construire une formule automatiquement

Une formule R peut être construite en deux étapes.

D'abord, on crée une formule sous forme de texte :

```
formule_texte <- paste(y, "~", premier_x)
```

Puis on transforme ce texte en vraie formule R :

```
formule_exemple <- as.formula(formule_texte)
```

Exemple

Si :

```
y <- "intention_achat"  
premier_x <- "prix_percu"
```

alors :

```
paste(y, "~", premier_x)
```

produit :

```
intention_achat ~ prix_percu
```

Puis :

```
as.formula(formule_texte)
```

transforme ce texte en formule utilisable par une fonction de modélisation.

Pourquoi utiliser `as.formula()` ?

Les fonctions de modélisation attendent souvent une formule R.

Par exemple :

```
FactoMineR::LinearModel(  
  formule_exemple,  
  data = questionnaire,  
  selection = "none"  
)
```

Ici, `formule_exemple` n'est plus seulement du texte.

C'est un objet formule.

Premier modèle sans boucle

Avant d'automatiser, la case ajuste un premier modèle :

```
modele_exemple <- FactoMineR::LinearModel(  
  formule_exemple,  
  data = questionnaire,  
  selection = "none"  
)
```

Cette étape permet de vérifier que la logique fonctionne pour une variable X.

On automatise seulement après avoir compris le cas simple.

Lire le premier modèle

Le modèle exemple contient notamment :

```
modele_exemple$Ftest  
modele_exemple$Ttest
```

Ces deux sorties permettent de distinguer :

Sortie	Rôle
Ftest	test global de l'effet
Ttest	coefficients du modèle

Cette distinction reste la même pour tous les modèles suivants.

Écrire une fonction

Quand une suite d'instructions doit être répétée, on peut l'enfermer dans une fonction.

La case crée :

```
ajuster_un_modele <- function(x) {  
  
  formule_texte <- paste(y, "~", x)  
  formule <- as.formula(formule_texte)  
  
  modele <- FactoMineR::LinearModel(  
    formule,  
    data = questionnaire,  
    selection = "none"  
  )  
  
  modele  
}
```

Rôle de la fonction

La fonction :

```
ajuster_un_modele()
```

reçoit le nom d'une variable explicative.

Elle construit automatiquement la formule correspondante, ajuste le modèle, puis retourne le résultat.

Par exemple :

```
ajuster_un_modele("prix_percu")
```

revient à ajuster :

```
intention_achat ~ prix_percu
```

Ce que la fonction généralise

La fonction généralise cette structure :

```
prendre une variable X  
→ construire la formule Y ~ X  
→ ajuster LinearModel()  
→ retourner le modèle
```

Le code devient plus court, mais surtout plus systématique.

Appliquer la fonction à toutes les variables

La fonction est ensuite appliquée à toutes les variables X avec :

```
modeles_univaries <- lapply(  
  variables_x,  
  ajuster_un_modele  
)
```

La fonction `lapply()` applique une même fonction à chaque élément d'un vecteur ou d'une liste.

Que fait `lapply()` ici ?

Ici, `lapply()` prend successivement :

```
prix_percu  
plaisir  
naturalite  
confiance  
ancrage_local  
usage_numerique  
sensibilite_env
```

et applique à chaque variable la fonction :

```
ajuster_un_modele()
```

Le résultat est une liste de modèles.

Nommer les éléments de la liste

La case ajoute ensuite des noms à la liste :

```
names(modeles_univaries) <- variables_x
```

Cela permet de retrouver facilement chaque modèle.

Par exemple :

```
modeles_univaries[["prix_percu"]]
```

désigne le modèle :

```
intention_achat ~ prix_percu
```

Objet créé : `modeles_univaries`

L'objet :

```
modeles_univaries
```

est une liste.

Chaque élément de cette liste est un modèle `LinearModel()`.

Élément	Modèle
prix_percu	intention_achat ~ prix_percu
plaisir	intention_achat ~ plaisir
naturalite	intention_achat ~ naturalite
confiance	intention_achat ~ confiance

Pourquoi stocker les modèles dans une liste ?

Une liste permet de conserver plusieurs résultats de même nature.

Ici, tous les éléments sont des modèles univariés.

Cela permet ensuite de :

- les afficher ;
- les comparer ;
- extraire leurs sorties ;
- les transformer en texte ;
- les utiliser dans un prompt.

Afficher les modèles de manière lisible

La case définit une fonction d’affichage :

```
afficher_modele_univarie <- function(nom_x, modele) {  
  cat("\n===== \n")  
  cat("Modèle univarié : ", y, " ~ ", nom_x, "\n", sep = "")  
  cat("Objet dans la liste : modeles_univaries[[\"", nom_x, "\"]] \n", sep = "")  
  cat("===== \n")  
  
  cat("\nFtest : \n")  
  print(modele$Ftest)  
  
  cat("\nTtest : \n")  
  print(modele$Ttest)  
}
```


Cette fonction sert à rendre la console plus lisible.

Pourquoi afficher seulement deux modèles ?

La case affiche seulement les deux premiers modèles.

L'objectif est pédagogique : comprendre la structure sans saturer la console.

Les autres modèles ne sont pas affichés, mais ils sont bien stockés dans :

```
modeles_univaries
```

On distingue donc l'existence d'un objet et son affichage dans la console.

Capturer les sorties sous forme de texte

La case transforme ensuite chaque modèle en texte :

```
textes_modeles_univaries <- lapply(
  modeles_univaries,
  function(modele) {
    paste(
      capture.output(print(modele)),
      collapse = "\n"
    )
  }
)
```

On retrouve ici une logique déjà vue dans les cases précédentes.

Rôle de `capture.output()`

La fonction :

```
capture.output()
```

capture ce qui aurait été imprimé dans la console.

Ici, elle transforme chaque sortie de modèle en texte.

Cela permet de préparer les résultats pour une utilisation ultérieure dans un prompt ou une fonction d'interprétation.

Rôle de `paste(..., collapse = "\n")`

`capture.output()` produit plusieurs lignes.

La fonction :

```
paste(..., collapse = "\n")
```

rassemble ces lignes dans une seule chaîne de caractères.

Chaque modèle devient donc un texte structuré.

Objet créé : `textes_modeles_univaries`

L'objet :

```
textes_modeles_univaries
```

est une liste de textes.

Chaque élément contient la sortie complète d'un modèle.

Élément	Contenu
<code>prix_percu</code>	sortie texte du modèle <code>intention_achat ~ prix_percu</code>
<code>plaisir</code>	sortie texte du modèle <code>intention_achat ~ plaisir</code>
<code>naturalite</code>	sortie texte du modèle <code>intention_achat ~ naturalite</code>

Pourquoi produire des textes ?

Les textes capturés sont utiles pour la suite du workflow.

Ils peuvent être :

- relus ;
- comparés ;
- injectés dans un prompt ;
- transmis à un outil d'aide à l'interprétation.

On transforme donc une série de modèles statistiques en matériau textuel réutilisable.

Ce que montre cette case

Cette case montre un passage important :

```
analyse unique  
→ fonction  
→ boucle  
→ liste de modèles  
→ liste de textes
```

C'est exactement la logique d'un workflow automatisé.

On ne répète pas le code à la main pour chaque variable.

Objets disponibles pour la suite

À la fin de la case, plusieurs objets sont disponibles :

Objet	Rôle
y	nom de la variable à expliquer
variables_x	noms des variables explicatives
formule_exemple	première formule construite automatiquement
modele_exemple	premier modèle ajusté sans boucle
ajuster_un_modele()	fonction qui ajuste un modèle pour une variable X
modeles_univaries	liste des modèles ajustés
textes_modeles_univaries	liste des sorties transformées en texte

Lien avec `condes()`

La case suivante introduira `condes()`.

L'idée est importante :

```
Nous venons de faire manuellement une série de modèles simples.  
condes() généralise cette logique pour décrire une variable quantitative.
```

Autrement dit, `condes()` automatise une partie du travail que cette case rend visible.

Ce qu'il faut retenir

Cette case n'est pas seulement une leçon sur `lapply()`.

Elle montre comment passer d'une analyse isolée à une logique systématique.

Le point important est :

```
si une opération est répétée plusieurs fois,  
elle peut devenir une fonction,  
puis être appliquée automatiquement.
```

Message clé

L'automatisation ne supprime pas la compréhension.

Au contraire, elle oblige à clarifier :

- ce qui reste fixe ;
- ce qui varie ;
- ce que l'on veut stocker ;
- ce que l'on veut afficher ;
- ce que l'on veut réutiliser.

```
Y ~ X  
→ Y ~ tous les X  
→ liste de modèles  
→ liste de sorties textuelles
```

Transition

Cette case prépare directement la suite.

Après avoir automatisé à la main plusieurs modèles univariés, on peut comprendre pourquoi des fonctions comme `condes()` sont utiles.

Elles généralisent cette logique pour produire une description systématique d'une variable quantitative.